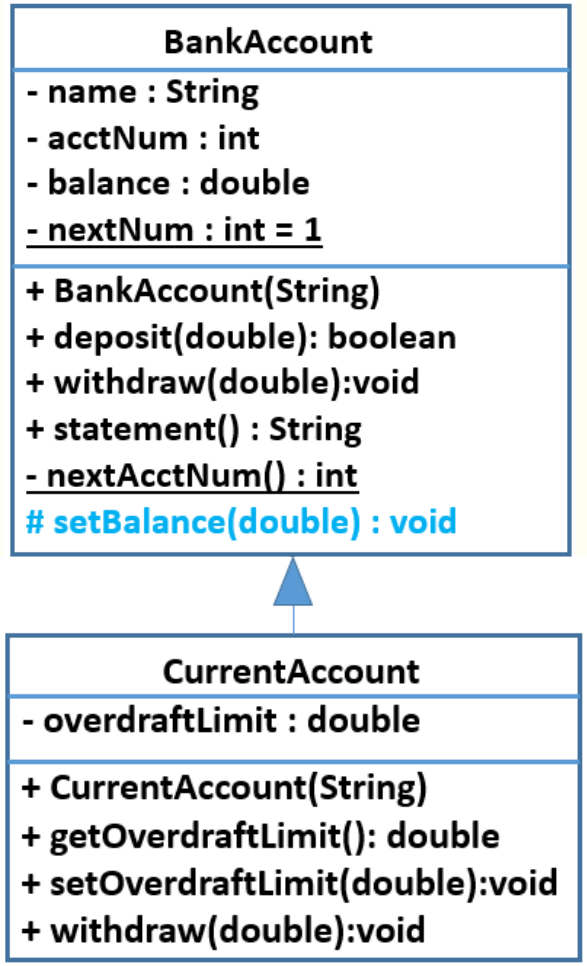
CSC1029 OO Programming

Lecture 19 – Runtime Polymorphism

QUEEN'S UNIVERSITY BELFAST

Recap on the Last lecture (1)




```
import bank.BankAccount; // import from package

BankAccount ba = new BankAccount("Paul"); // standard account

ba.deposit(20.0); // deposit £20

ba.withdraw(10.0); // withdraw £10

ba.setBalance(2000.0); // can't do this – method is 'protected'
                       // same visibility as 'private' for code
                       // defined in a separate package
```

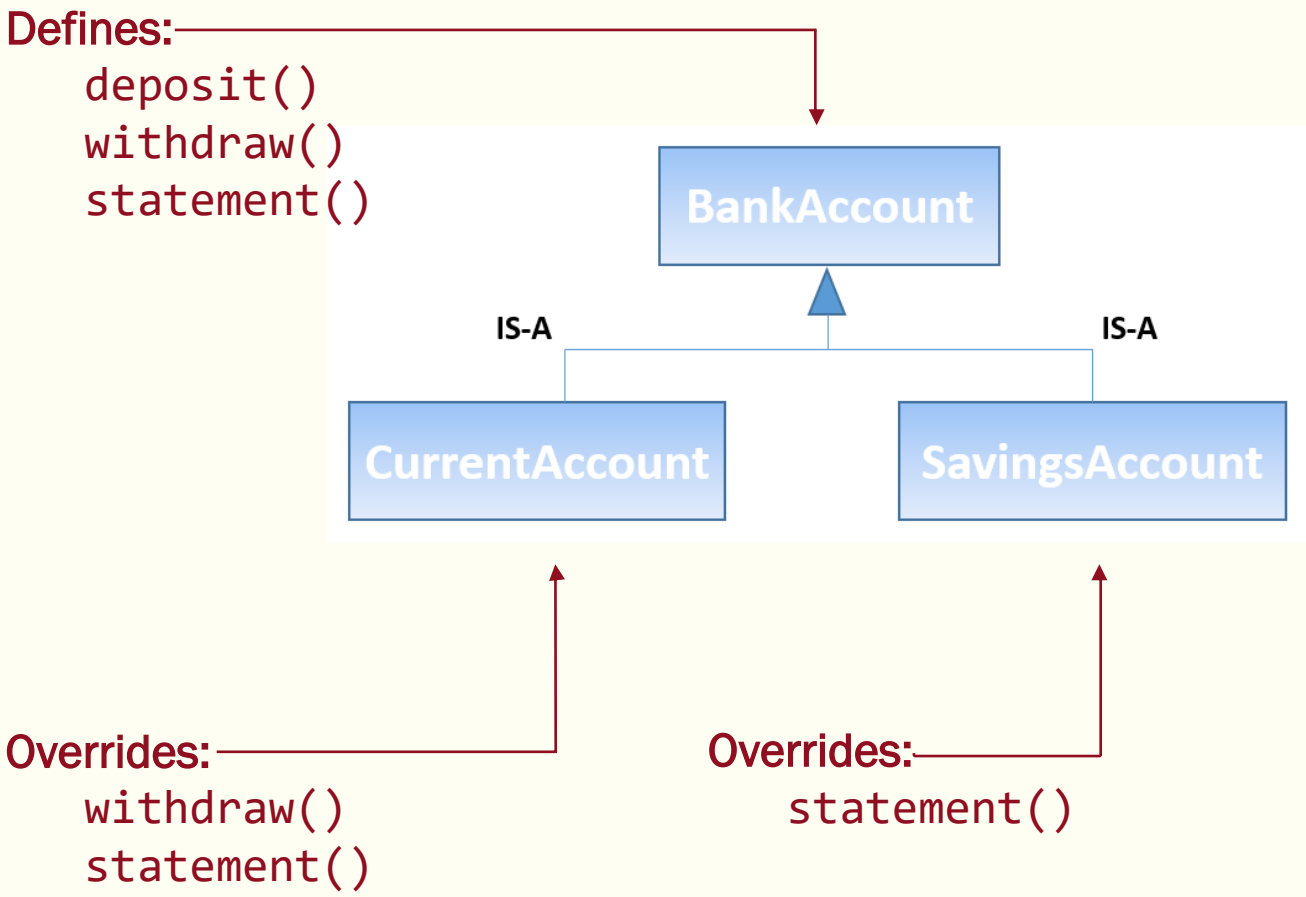
```
import bank.CurrentAccount; // import from package

CurrentAccount ca = new BankAccount("Fred"); // current account

ca.withdraw(1000.0); // withdraw £1000 – overridden method
// code for 'withdraw' can use 'setBalance' as protected methods
// are inherited
```

QUEEN'S UNIVERSITY BELFAST

Recap on the Last lecture (2)

A class may **extend** another class

- **single** parent class (superclass)
- parent constructor called through **super**
- **public** methods/data **visible** in subclass
- **private** methods/data **not inherited**
- **protected** methods/data are inherited
- can **override** methods in subclass



A Polymorphic Collection (1)

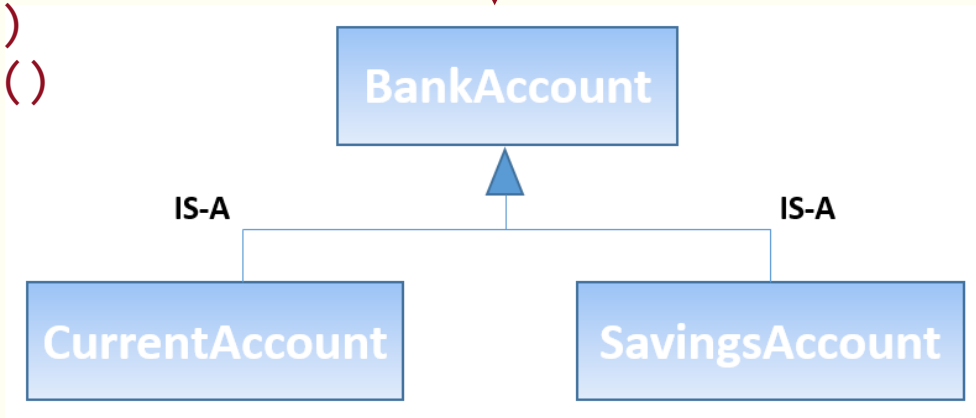
A Bank *Manages* Bank Accounts

General logic for quarterly fees

```
for each BankAccount in the Bank
   withdraw fees for the BankAccount
```

Suppose fees are £10 no matter what type of account.

Defines:
- deposit()
- withdraw()
- statement()



Overrides:
- withdraw()
- statement()

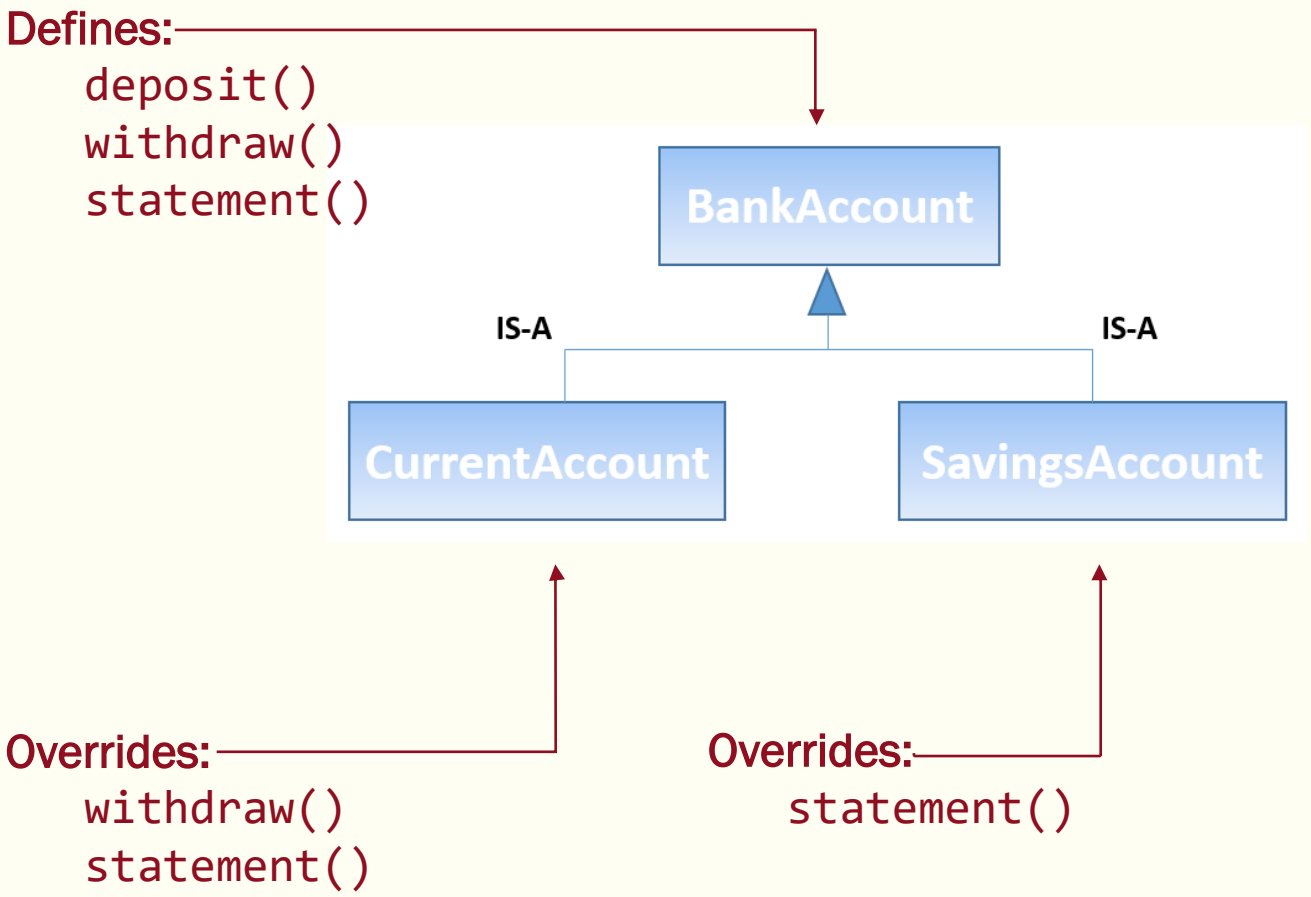
Overrides:
- statement()

QUEEN'S UNIVERSITY BELFAST

A Polymorphic Collection (2)

Instantiate some objects ...

```
BankAccount ba1 = new
        BankAccount(..);

SavingsAccount sa1 = new
        SavingsAccount(..);

CurrentAccount ca1 = new
        CurrentAccount(..);

ba1.deposit(20.0);
sa1.withdraw(10.0);
ca1.withdraw(10.0);
```



A Polymorphic Collection (3)

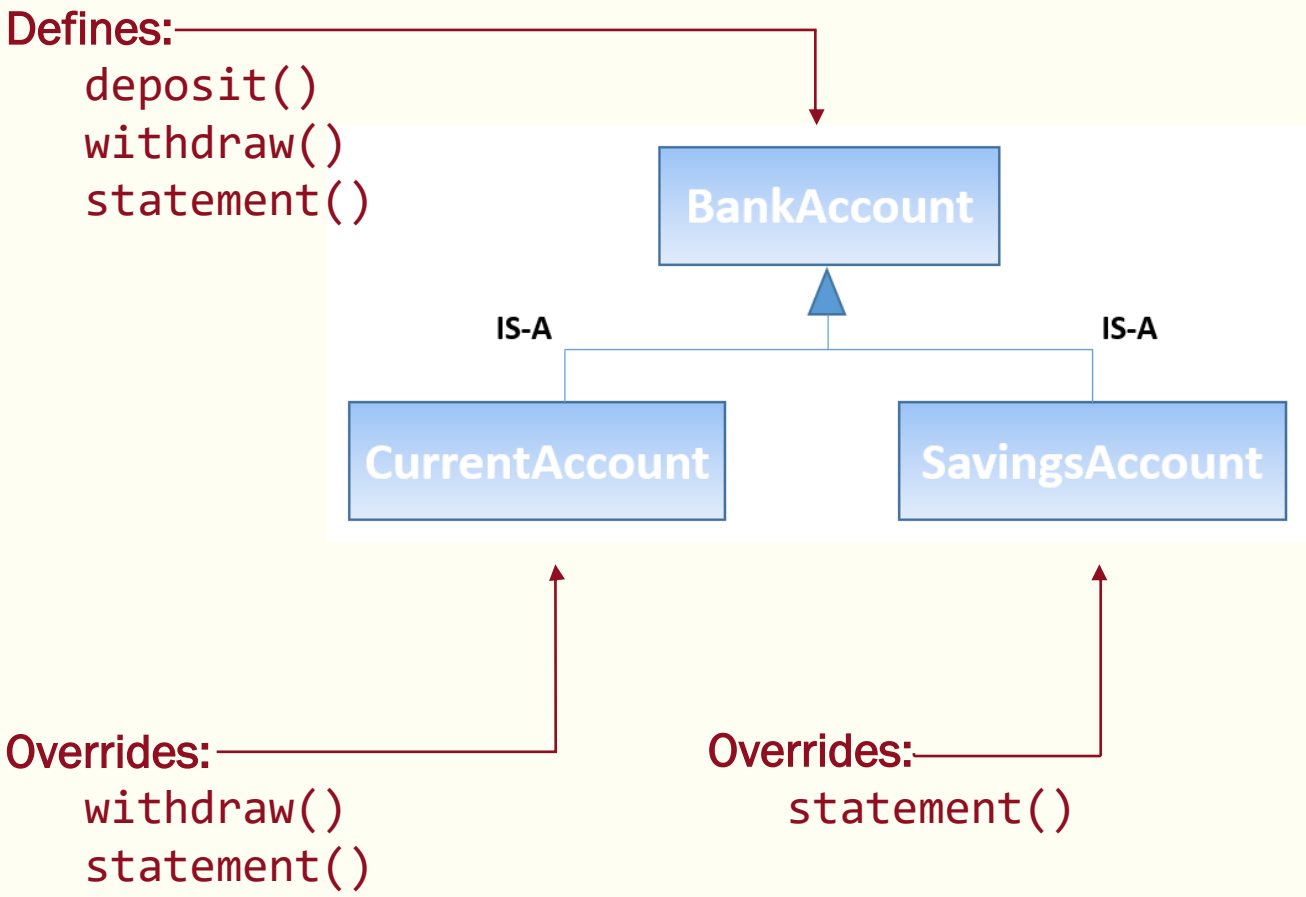
Instantiate a collection ...

```
ArrayList<BankAccount> theBank =
        new ArrayList<BankAccount>();

theBank.add(ba1); // BankAccount
theBank.add(sa1); // SavingsAccount
theBank.add(ca1); // CurrentAccount
```

'theBank' is a ***polymorphic*** structure.

It contains objects of ***different*** types ...

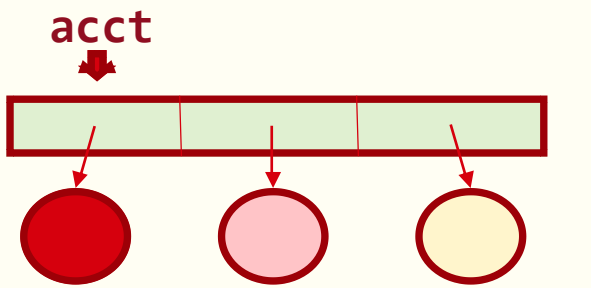
... but they are all types of ***BankAccount***



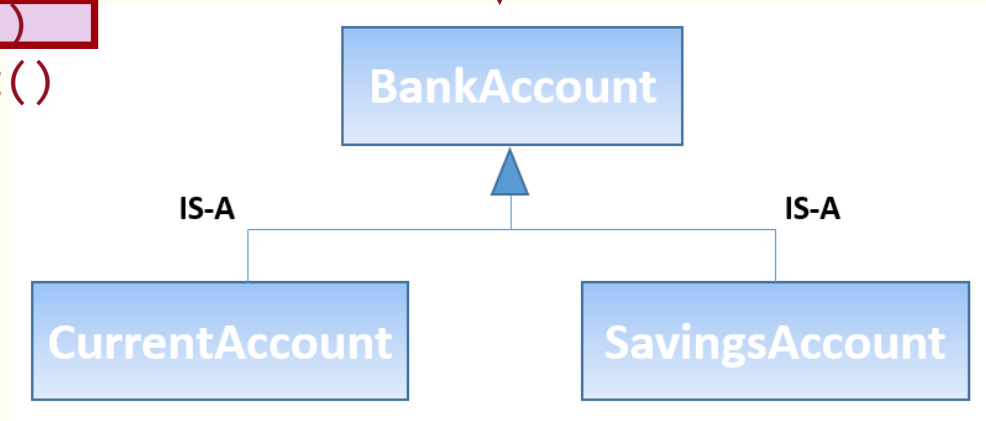
A Polymorphic Collection (4)

Instantiate a collection ...

```
ArrayList<BankAccount> theBank =
        new ArrayList<BankAccount>();

for(BankAccount acct : theBank) {
        acct.withdraw(10.0);
        String s = acct.statement();
}
```



Defines:
- deposit()
- withdraw()
- statement()



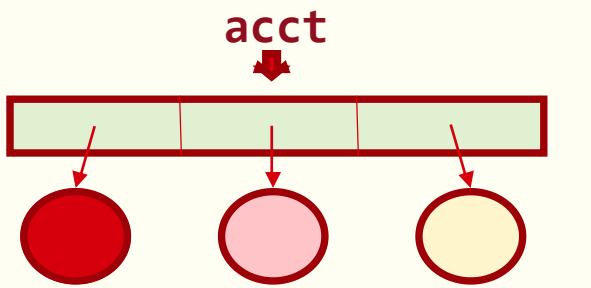
Overrides:
- withdraw()
- statement()

Overrides:
- statement()

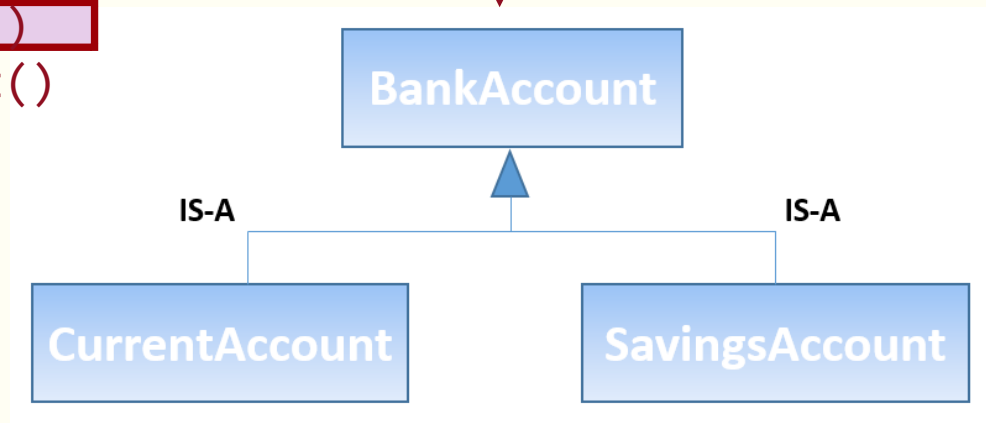
A Polymorphic Collection (5)

Instantiate a collection ...

```
ArrayList<BankAccount> theBank =
        new ArrayList<BankAccount>();

for(BankAccount acct : theBank) {
        acct.withdraw(10.0);
        String s = acct.statement();
}
```



Defines:
- deposit()
- withdraw()
- statement()



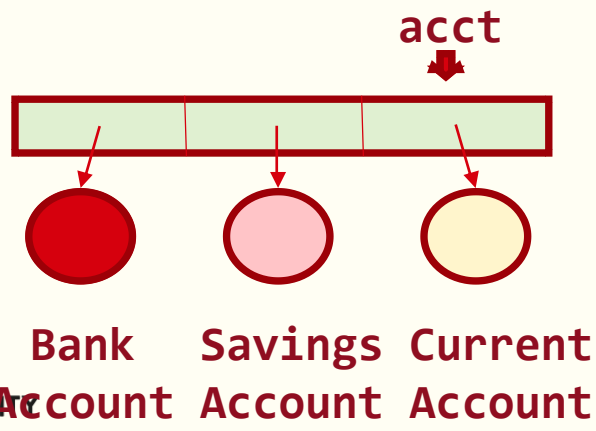
Overrides:
- withdraw()
- statement()

Overrides:
- statement()

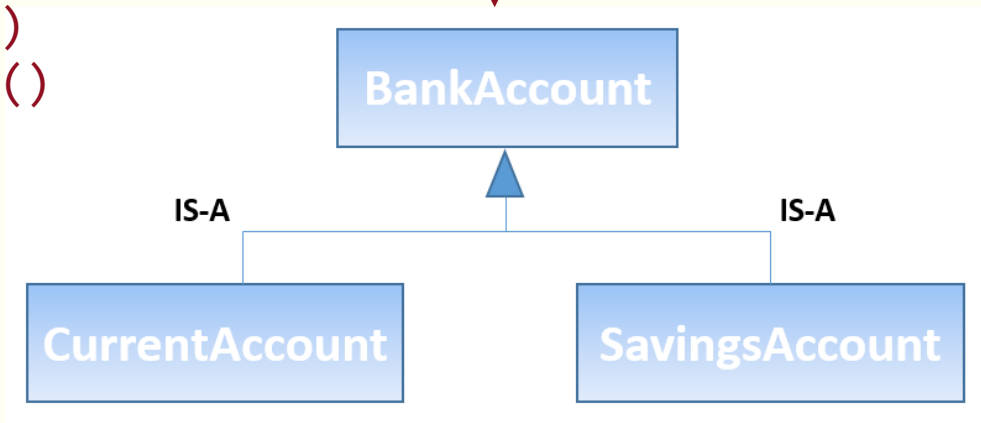
A Polymorphic Collection (6)

Instantiate a collection ...

```
ArrayList<BankAccount> theBank =
        new ArrayList<BankAccount>();

for(BankAccount acct : theBank) {
        acct.withdraw(10.0);
        String s = acct.statement();
}
```



Defines:
deposit()
withdraw()
statement()



Overrides:
withdraw()
statement()

Overrides:
statement()

Abstract Classes (1)

Consider the following classes representing different types of **Shape**:

- Each could be implemented separately, with no associations
- Each has a separate definition for calculating shape area, spending on the actual shape
- However, the classes are logically related in the sense that they are all types of **Shape**

Rectangle
-width : double -height : double
+constructors… +getters… +setters… +getArea():double

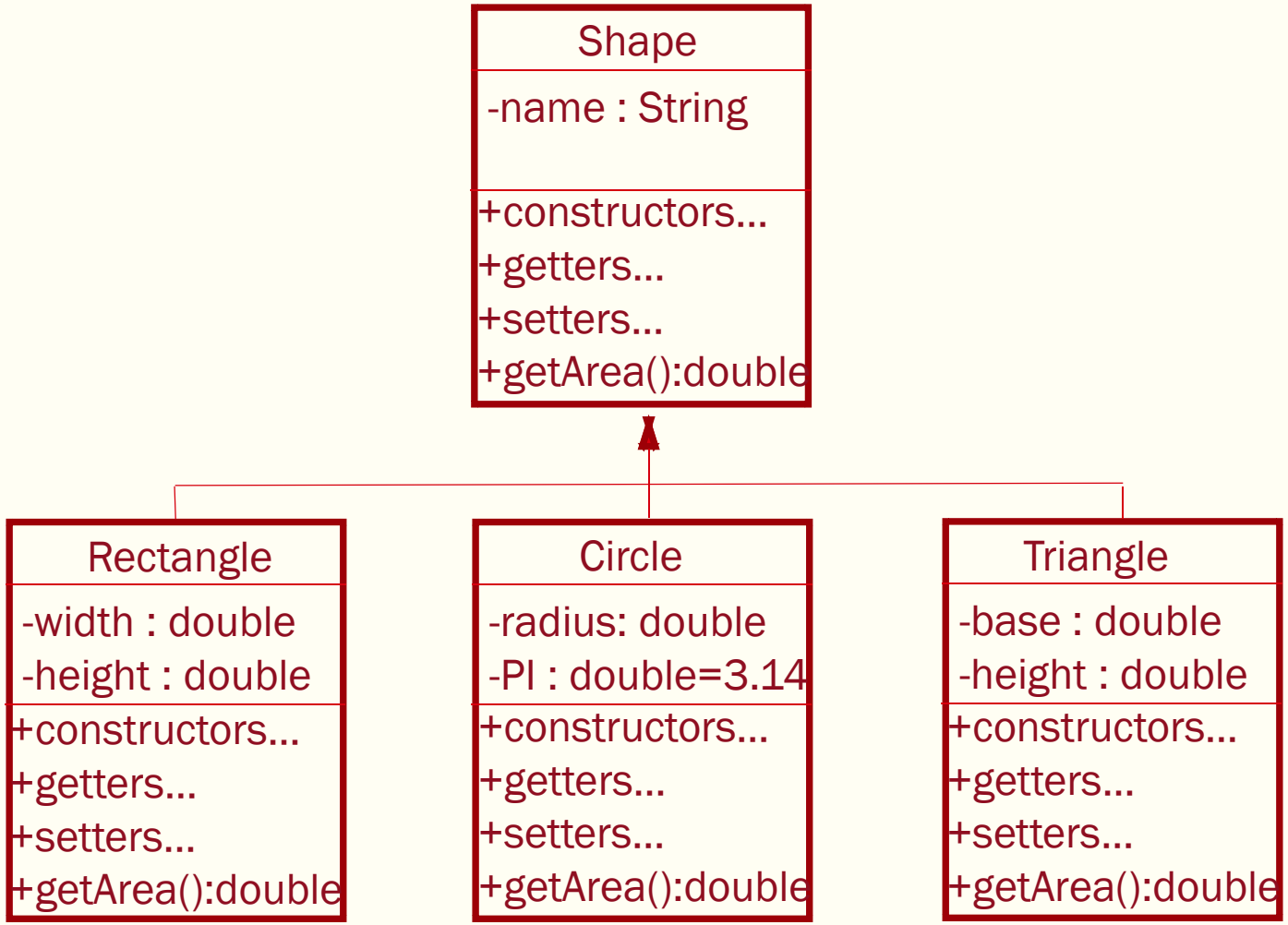
Circle
-radius: double -PI : double=3.14
+constructors… +getters… +setters… +getArea():double

Triangle
-base : double -height : double
+constructors… +getters… +setters… +getArea():double

Would it make any sense to define a **superclass** called **Shape** to define this relationship?

What would be the implications of doing this?

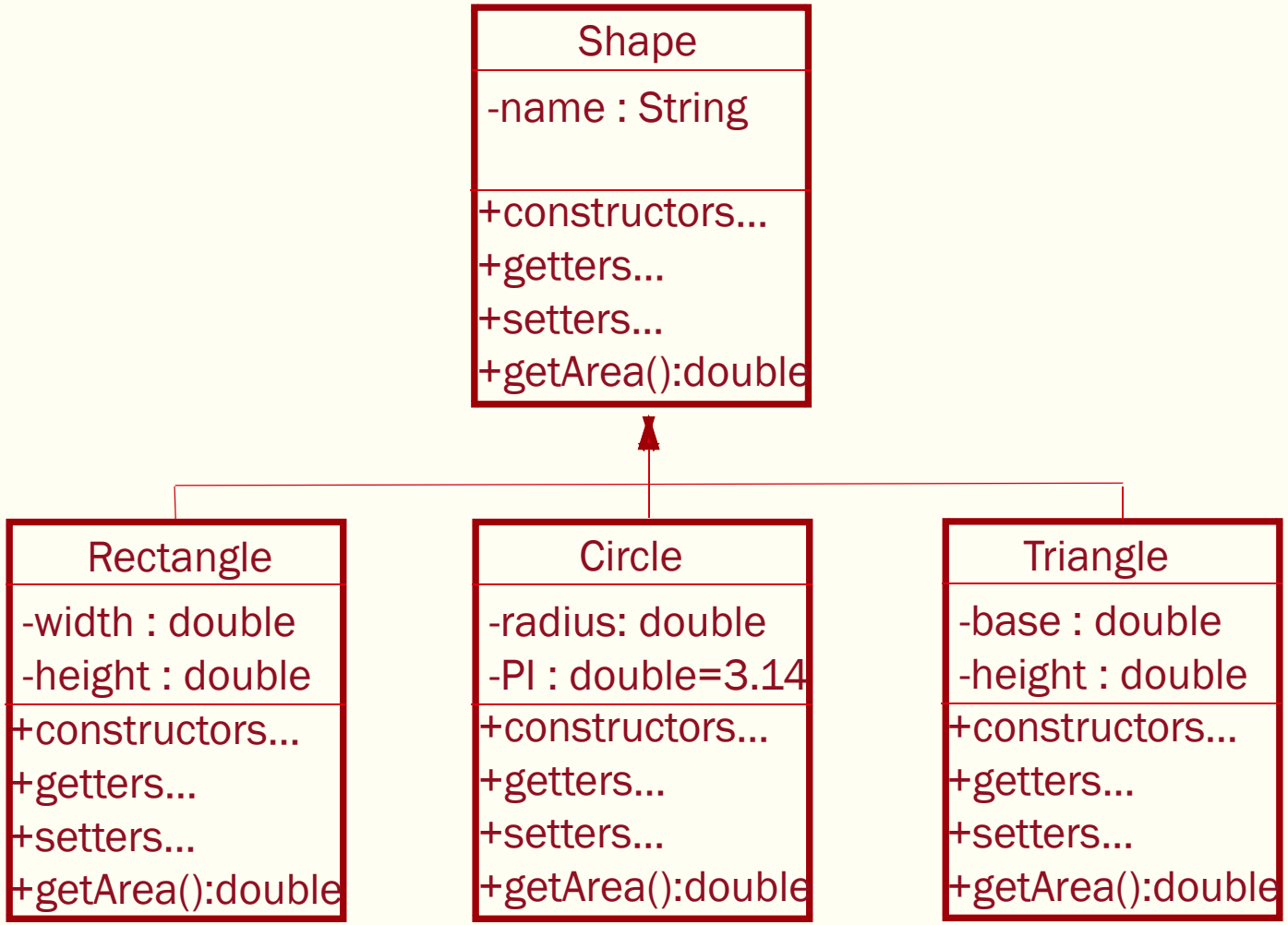
Abstract Classes (2)



Q1. Can we define class ***Shape*** with a ***getArea***() method then override this definition to be more specific in the subclasses?

Q2. What would the point of this be?

Abstract Classes (3)

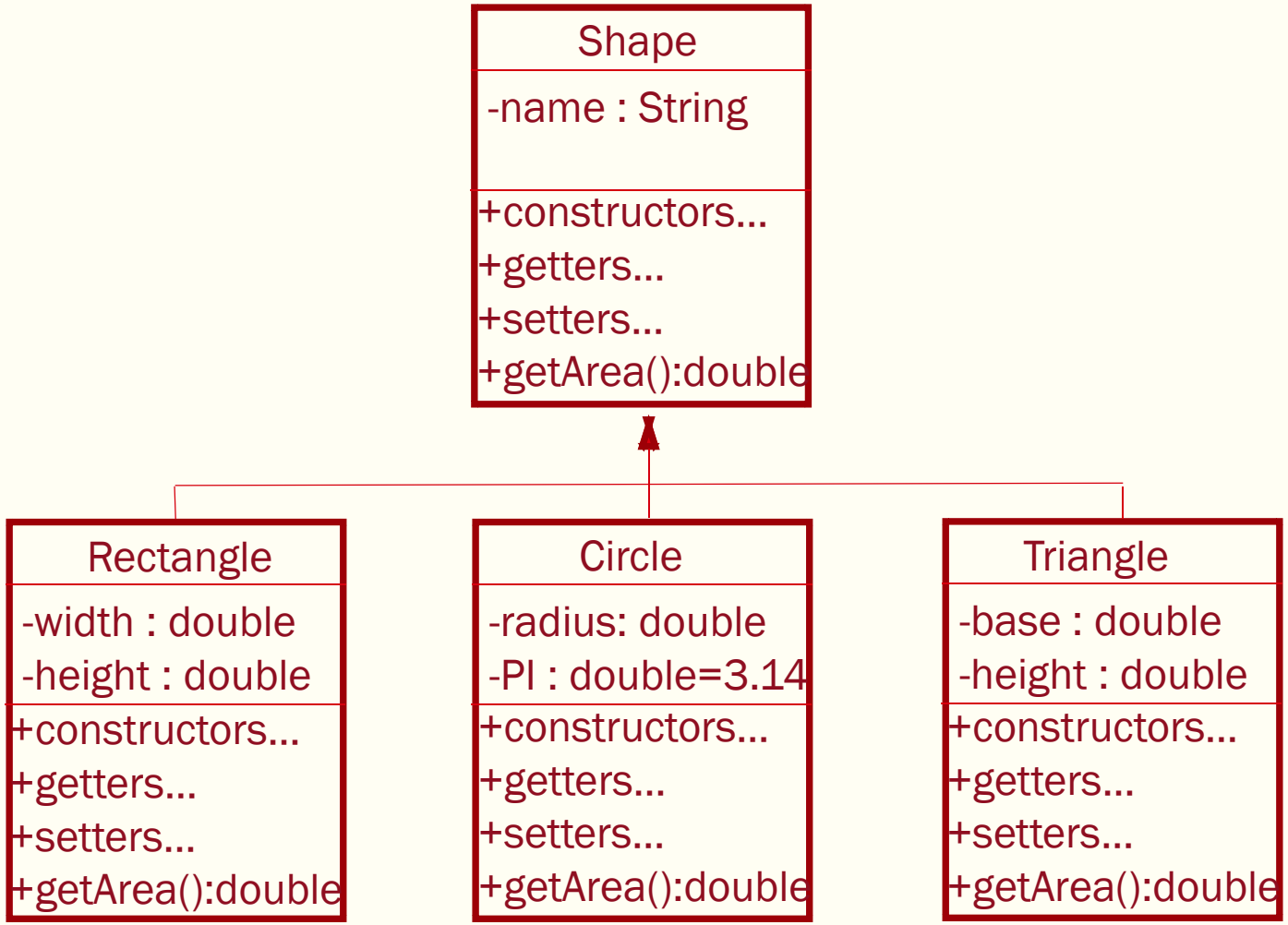


Q1. Can we define class ***Shape*** with a ***getArea***() method then override this definition to be more specific in the subclasses?

But – the problem here is that is makes no sense to get the area of a ***Shape*** in general.

At this level – this is an ***abstract*** concept!

Abstract Classes (4)

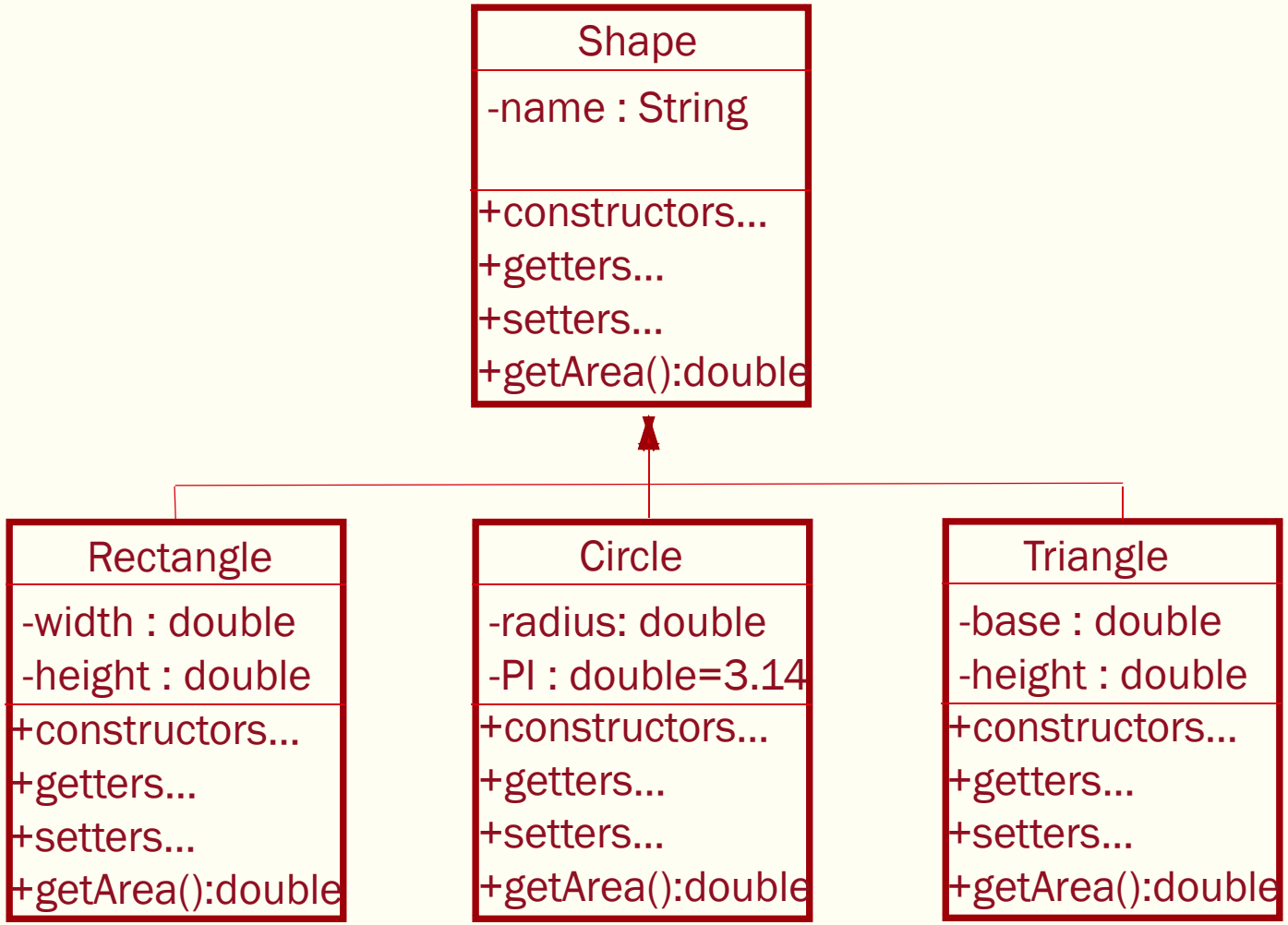


Since the method ***getArea***() can't be implemented within ***Shape*** – we can declare the method as ***abstract*** – and omit any implementation.

Any class that contains an ***abstract*** method must also be declared as ***abstract***.

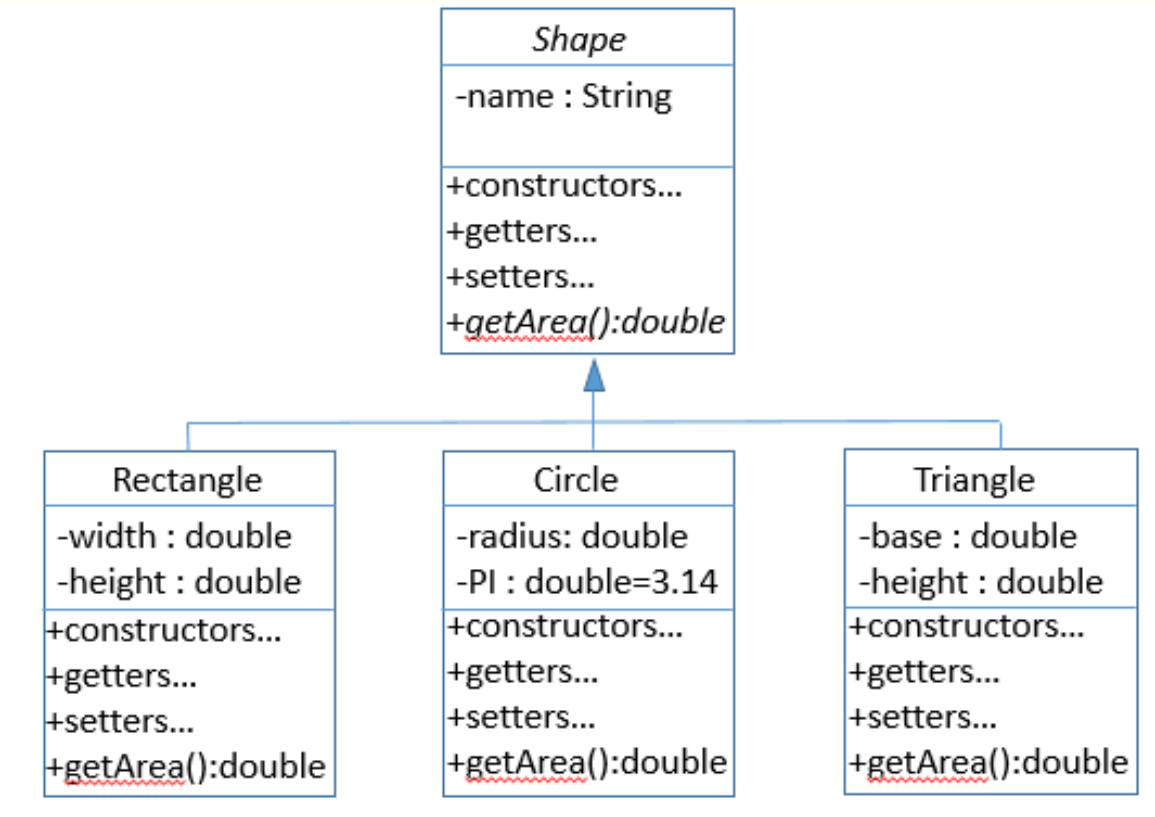
Abstract classes cannot be used to instantiate objects

Abstract Classes (5)



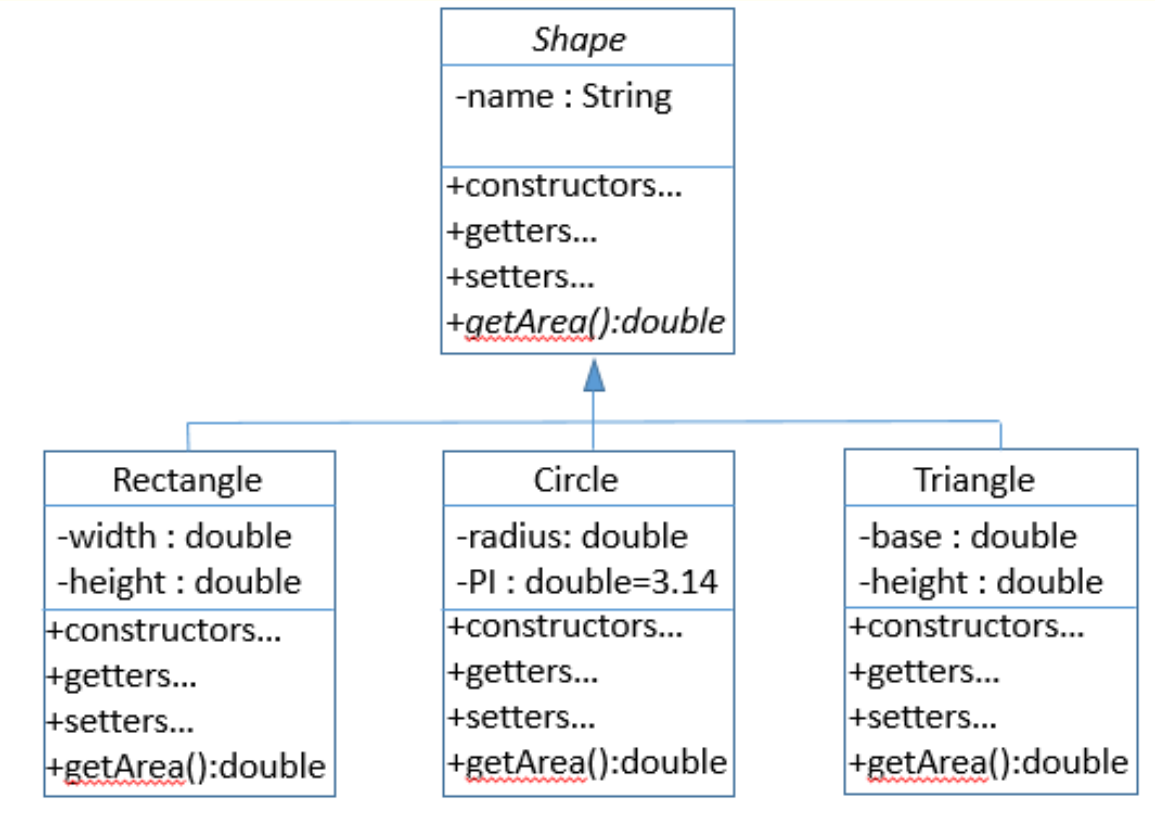
In UML the name of ***abstract*** classes and methods are in *italics*.

Abstract Classes (6)



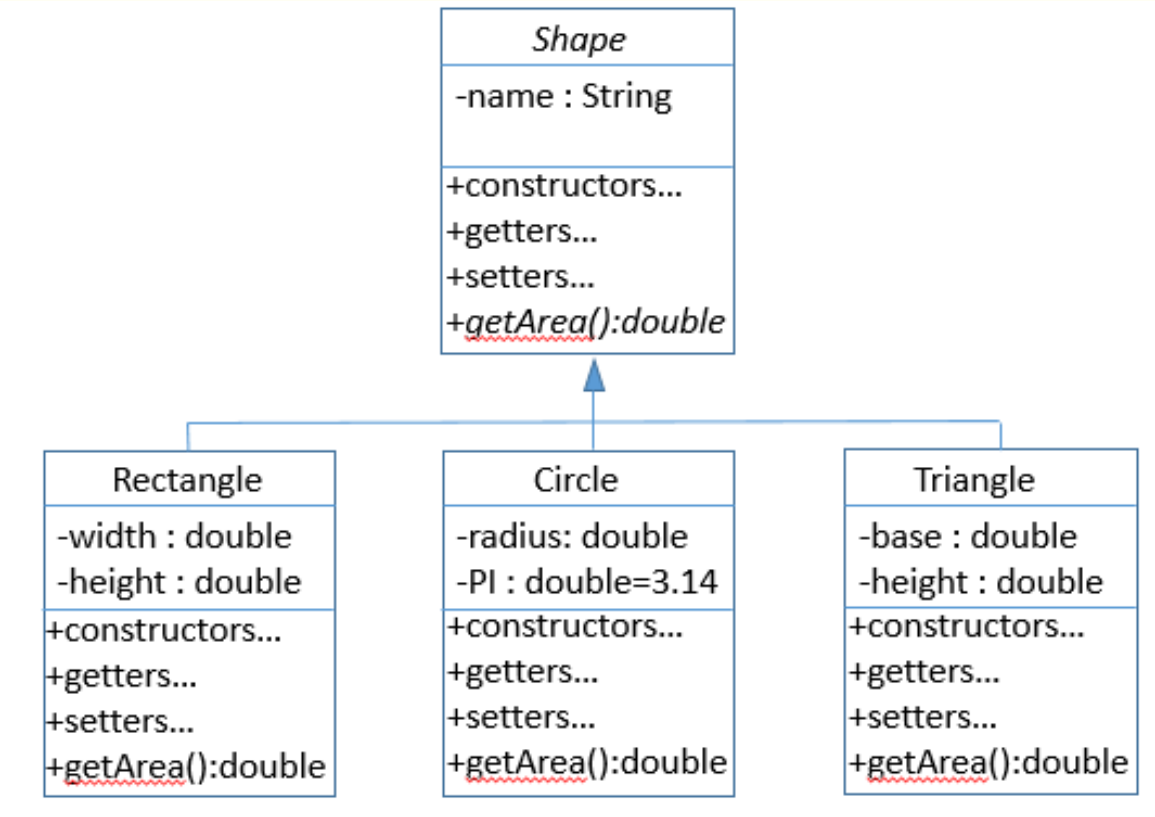

```
public abstract class Shape {
    public String name;

    public Shape(String name) {
        if (name != null) {
            this.name = name;
        }
        else {
            this.name = "Unknown Shape";
        }
    }

    public abstract double getArea();

}
```

Abstract Classes (7)




```
public class Circle extends Shape {
    private final double PI = 3.14;
    private double radius;

    public Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    @Override
    public double getArea() {
        return PI*radius*radius;
    }
}
```

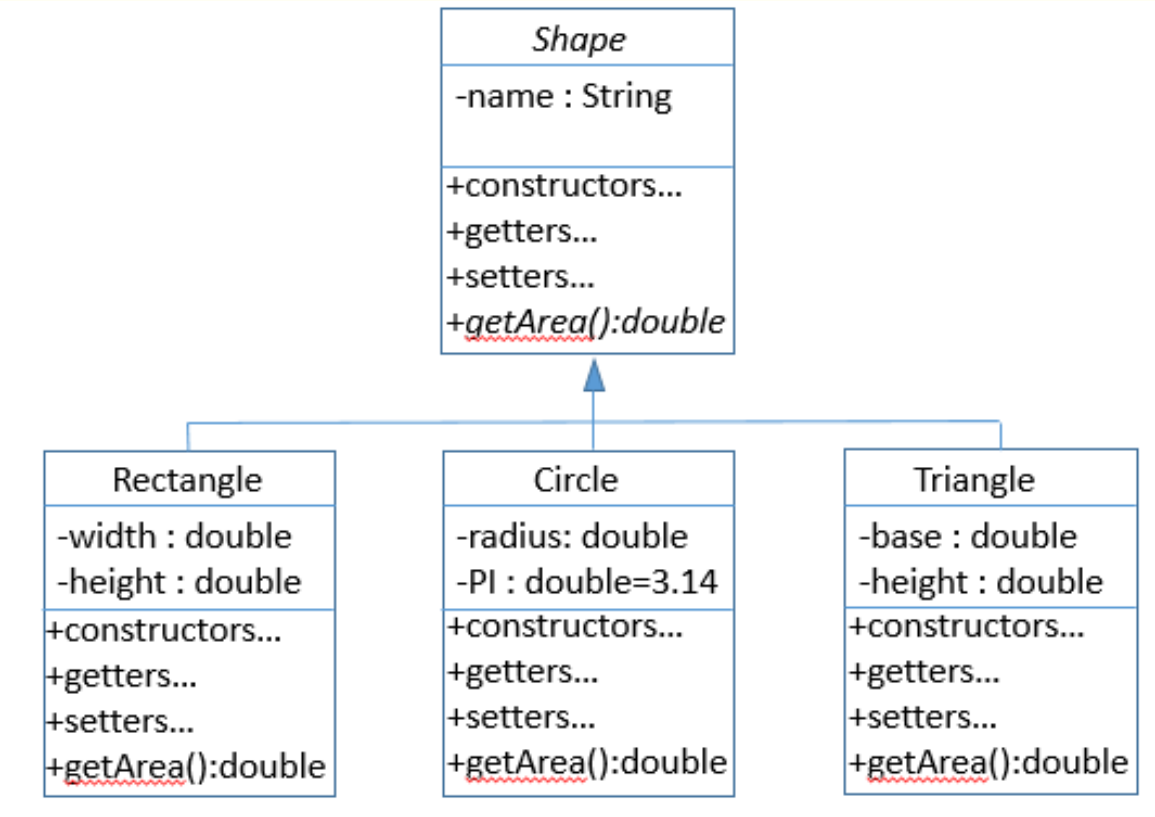
Abstract Classes (7)




```
public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(double width, double
                                height) {
        super("Rectangle");
        this.width = width;
        this.height = height;
    }

    @Override
    public double getArea() {
        return this.width * this.height;
    }
}
```

QUEEN'S UNIVERSITY BELFAST

Abstract Classes (8)

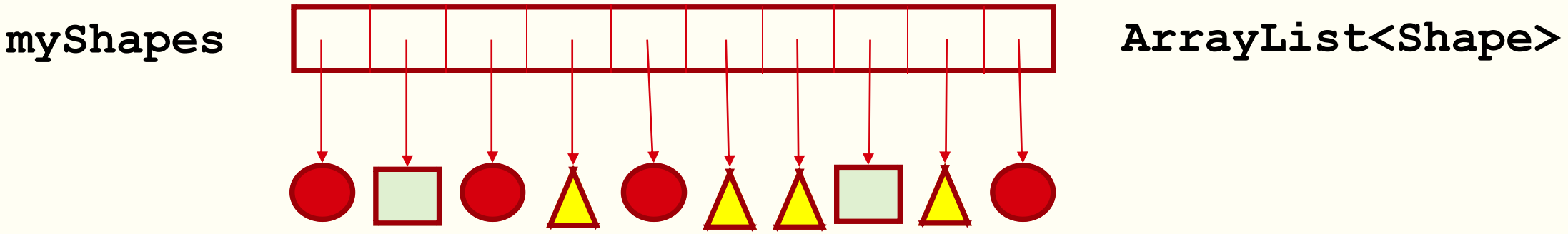



```
public class Triangle extends Shape {
    private double base;
    private double height;

    public Triangle(double base, double
                                    height) {
        super("Triangle");
        this.base = base;
        this.height = height;
    }

    @Override
    public double getArea() {
        return 0.5 * base * height;
    }
}
```

The JDK will ***force implementation*** of ***abstract methods*** in classes that ***extend*** the behaviour of ***abstract classes***.

Runtime Polymorphism (1)

Q2. What would the point of this be? Consider the following situation:

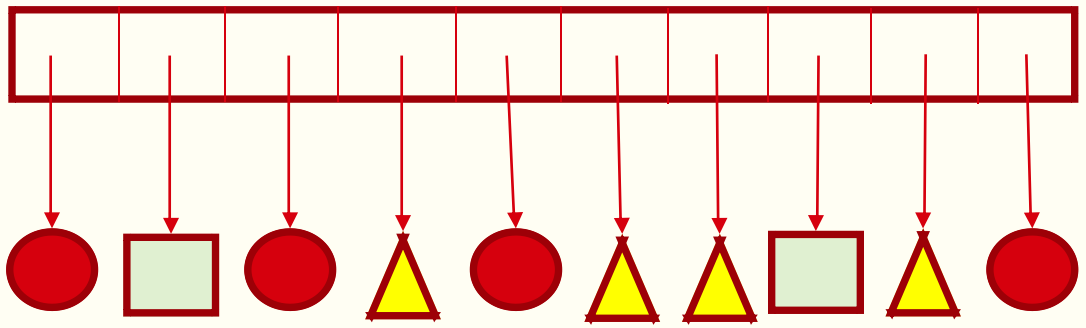


We can maintain a collection of Shape references – instances of different subclasses.

Runtime Polymorphism (2)

Q2. What would the point of this be? Consider the following situation:

myShapes

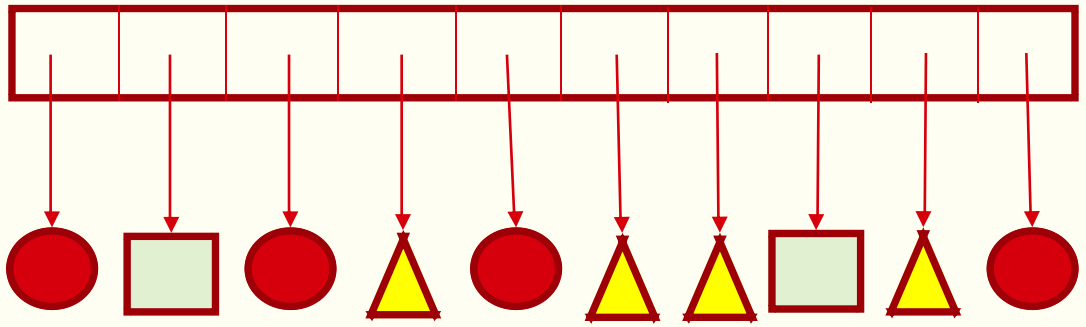


ArrayList<Shape>

```
for(Shape s : myShapes) {
    System.out.println("\tArea: " + s.getArea());
}
```

Using a superclass reference to a subclass instance – method called will depend on instance – this decision is not made until runtime.

Runtime Polymorphism (3)




```
for(Shape s : myShapes) {
    System.out.println("\tArea: " + s.getArea());
}
```

For some future class X .. That *extends Shape* .. As long as it overrides *getArea()* .. The code above will work ***without*** change .. Even without recompilation.

Interfaces (1)

Q: How do we force a class to implement a method?

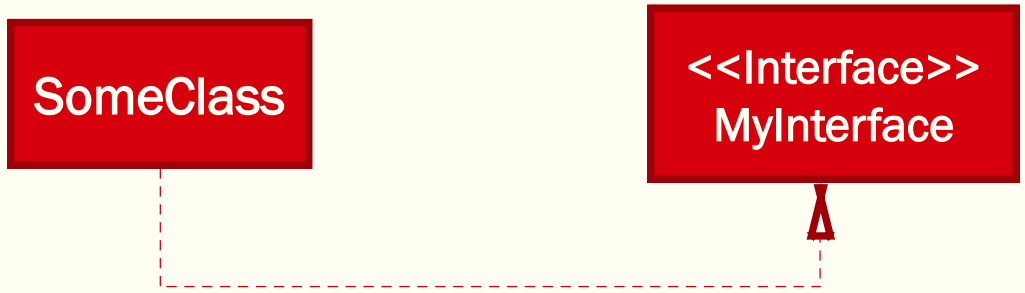
Q: Why would you want to do this?

A: To ensure that a class contains methods (specifically defined) that are required by clients (other code) that need to use the class.

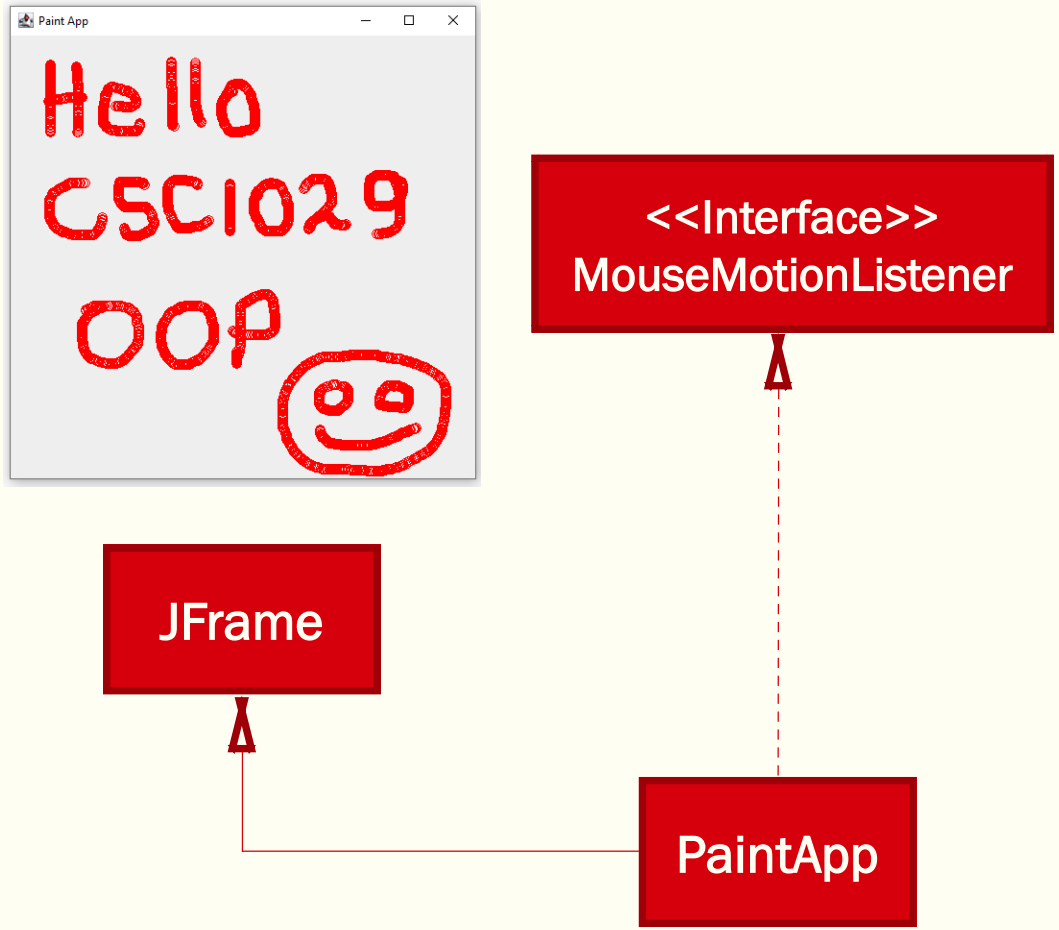
A good mechanism when working in a team – a way of agreeing available behaviour.

An interface defines a named list of method signatures….

```
public interface MyInterface {
    public void method1(int number);
    public int method2();
    public String method3(int number);
}
```



Interfaces (2)




```
public class PaintApp extends JFrame implements
                MouseMotionListener {

   public PaintApp(String text) { ... }

   public static void main(String[] args) {
      new PaintApp("Paint App");
   }

   public void mouseDragged(MouseEvent me) {
      Graphics g = this.getGraphics();
      g.setColor(Color.RED);
      g.drawOval(me.getX(), me.getY(), 10, 10);
   }

   public void mouseMoved(MouseEvent me) { ... }

}
```

What's Next?

More on ...

- Inheritance
- Interfaces
- Object References